Защита КР №3: что в проекте за что отвечает

Backend «Киностудия / FrameSpace» (заявки на съёмку) — серверное приложение на Spring Boot. Оно хранит данные в PostgreSQL, создаёт схему миграциями Flyway, защищает API через Spring Security и JWT и запускается в Docker Compose. Основа — база КР №1: клиенты, локации и заявки с тем же графом статусов, тестовые данные КР №1 загружаются миграцией. Этим REST API будет пользоваться Android-приложение из КР №4. JavaFX-клиент из КР №2 остаётся отдельной программой.

1. Общая схема

```
Клиент (web UI, Swagger, Android)
   →  Фильтры        →  Controller   →  Service   →  Repository   →  PostgreSQL
      web, security     controller      service      repository       таблицы (Flyway)
                        dto (вход/выход)              domain (JPA-сущности)
```

- **Фильтры** срабатывают до контроллера: присваивают запросу correlation ID, проверяют JWT, не пускают пользователя с временным паролем дальше страницы смены пароля.
- **Controller** принимает HTTP-запрос, проверяет DTO (`@Valid`) и роль (`@PreAuthorize`), вызывает сервис и возвращает DTO. Бизнес-правил и запросов к базе в нём нет.
- **Service** содержит правила предметной области и транзакции (`@Transactional`): граф статусов, запрет даты в прошлом, расчёт приоритета, архивирование, проверка версии, массовые операции, импорт CSV, запись в аудит.
- **Repository** — интерфейсы Spring Data JPA. Простые запросы строятся по имени метода, сложные — через `@Query` и Criteria API (`Specification`).
- **PostgreSQL** хранит данные и ограничения: первичные и внешние ключи, `NOT NULL`, `UNIQUE`, `CHECK`, индексы.

Правило, которое стоит озвучить: адрес и права доступа — в контроллере; формат входных данных — в DTO; правило предметной области — в сервисе; запрос — в репозитории; структура таблиц — только в миграции Flyway. JPA-сущность клиенту никогда не возвращается, только DTO.

2. Пошагово: что происходит при запуске

1. Пользователь копирует `.env.example` в `.env` в корне проекта и вписывает свои пароли и JWT-секрет.
2. `backend/start-compose.ps1` (Windows) или `backend/start-compose.sh` (macOS, Linux) выполняет `docker compose --env-file .env -f backend/docker-compose.yml up -d --wait`.
3. Сервис `db` поднимает PostgreSQL 17. Его healthcheck (`pg_isready`) сообщает, когда база готова принимать подключения.
4. Сервис `backend` ждёт здоровую базу (`depends_on: condition: service_healthy`) и запускает Spring Boot через Maven-образ с JDK 21.
5. При старте Spring Boot **Flyway** применяет миграции из `db/migration`: V1 — схема, V2 — клиенты и тип локации, V3 — тестовые данные КР №1. Затем Hibernate с `ddl-auto: validate` сверяет JPA-сущности со схемой и сам таблицы не меняет.
6. `BootstrapAdmin` создаёт первого администратора, если таблица пользователей пуста. Логин и временный пароль берутся из `.env`; пароль в журнал не пишется.
7. Healthcheck backend проверяет, что Tomcat слушает порт 8080. Сервис `connection-info` печатает адреса web-интерфейса, Swagger и базы.

Что происходит при одном запросе. `CorrelationIdFilter` → `JwtAuthenticationFilter` (подпись, issuer, срок, роли из базы) → `PasswordChangeRequiredFilter` → проверка роли в `@PreAuthorize` → контроллер →

сервис → репозиторий → ответ DTO в JSON. Любое исключение по пути превращается в единый JSON `ApiError` в `GlobalExceptionHandler`.

3. Файлы по слоям

Код находится в `backend/src/main/java/ru/mirea/framespace/`. Перенос с примера Student Hub: `Student` → `ShootingRequest`, `StudyGroup` → `Location`, `StudentStatus` → `RequestStatus`, `RiskLevel` → `PriorityLevel`, `StudentNote` → `RequestNote`, `student_tag` → `request_tag`, роль `TEACHER` → `OPERATOR`. Сверх примера добавлен `Client` — таблица клиентов из КР №1.

3.1 REST-контроллеры: `controller/`

Класс	Адреса	Кто может
`AuthController`	`/api/auth/login`, `refresh`, `logout`, `me`, `change-password`, `change-initial-password`	login, refresh, logout — без входа; остальное — любой вошедший
`RequestController`	`/api/requests`: список, карточка, создание, изменение, архив, `/restore`, `/permanent`, `/bulk`, `/import/preview`, `/import/apply`, `/export`	чтение — все роли; изменение, bulk, CSV — `OPERATOR`, `ADMIN`; окончательное удаление — `ADMIN`
`ClientController`	`/api/clients`: список с поиском, карточка, создание, изменение, удаление	чтение — все роли; создание и изменение — `OPERATOR`, `ADMIN`; удаление (только клиента без заявок) — `ADMIN`
`NoteController`	`/api/requests/{requestId}/notes`: список, создание, изменение, удаление	`OPERATOR`, `ADMIN`
`LocationController`	`/api/locations`: список, создание, изменение, `/{id}/active`	список — все роли; изменение справочника — `ADMIN`
`AnalyticsController`	`/api/analytics/summary`	все роли
`AdminUserController`	`/api/admin/users`: список, создание, изменение, `/{id}/reset-password`	`ADMIN`
`AuditController`	`/api/admin/audit`: поиск, сортировка, пагинация	`ADMIN`

Слова `items` и `references` из задания заменены понятиями темы: `requests` (заявки), `clients` (клиенты) и `locations` (локации).

3.2 Бизнес-логика: `service/`

Класс	Что делает
`RequestService`	Поиск с пагинацией, создание и изменение заявки, архивирование вместо удаления, восстановление, окончательное удаление, массовые операции. Правила КР №2: новая заявка — только в статусе `NEW`; дата съёмки не в прошлом (при создании и при смене даты); клиент и локация существуют, локация активна; статус меняется только по графу (иначе 409). Также: сверка `version`, уникальный внешний идентификатор, не больше 20 тегов. После каждого изменения пересчитывает приоритет и пишет событие в аудит.

Класс	Что делает
`ClientService`	Клиенты: поиск по ФИО, email и телефону, email уникален без учёта регистра, проверка версии. Клиента со связанными заявками удалить нельзя (409). Для CSV-импорта находит клиента по email или создаёт нового.
`PriorityCalculator`	Вычисляемый показатель. Завершённая, отклонённая или отменённая заявка — `NOT_APPLICABLE`. Срочная с бюджетом от 500 000 — `CRITICAL`. Срочная, бюджет от 500 000 или длительность от 16 часов — `HIGH`. Бюджет от 200 000 или длительность от 8 часов — `MEDIUM`. Иначе — `LOW`. Клиент приоритет не присылает: его считает сервер.
`RequestMapper`	Переводит сущность `ShootingRequest` в `RequestResponse` внутри транзакции.
`LocationService`	Справочник локаций: код уникален и приводится к верхнему регистру, включение и отключение без физического удаления, проверка версии.
`NoteService`	История заметок по заявке (закреплённые сверху). Изменить или удалить заметку может только её автор или администратор.
`CsvRequestService`	Экспорт текущей выборки частями по 500 строк (UTF-8 с BOM). Проверка файла без записи в базу — и формата, и бизнес-правил (статус новой строки, переход статуса, дата) — и отдельное применение импорта. Клиент строки ищется по email, новый создаётся. Режимы `CREATE_ONLY` и `UPSERT_BY_EXTERNAL_ID`; ограничение 5 000 строк; защита от формул Excel; выгруженный файл можно загрузить обратно.
`AnalyticsService`	Сводка: всего, активные, архивные, срочные, средний бюджет, группировки по локации, статусу и приоритету. Считает база через запросы с `count`, `avg`, `group by`.
`AuthService`	Вход, выдача access JWT и refresh-токена, ротация refresh-токена, выход, смена пароля и первичная замена временного пароля.
`LoginAttemptService`	После 5 неудачных попыток блокирует вход на 5 минут; каждая следующая неудача увеличивает блокировку, максимум до 40 минут. Счётчик пишется в отдельной транзакции, чтобы откат запроса его не сбрасывал.
`UserAdminService`	Пользователи с пагинацией; создание с временным паролем; изменение имени, логина, ролей и активности; сброс пароля. Отключение пользователя отзывает его сессии. Администратор не может отключить себя или снять с себя роль `ADMIN`.
`AuditService`	Записывает событие в отдельной транзакции: пользователь, тип события, тип сущности, UUID, время, описание, correlation ID, IP. Поиск по журналу.

3.3 Работа с базой: `repository/`

- `ShootingRequestRepository` расширяет `JpaRepository` и `JpaSpecificationExecutor`: обычный CRUD, поиск по `Specification`, счётчики и группировки для аналитики (`@Query` на JPQL с параметрами).
- `ClientRepository`: поиск клиента по email без учёта регистра, поиск по подстроке с пагинацией.
- `LocationRepository`, `RequestNoteRepository`, `UserAccountRepository`, `RefreshSessionRepository`, `AuditEventRepository` — аналогично для своих таблиц.
- SQL вручную не собирается: Spring Data строит запросы по именам методов, значения передаются параметрами.

3.4 Сущности: `domain/`

Класс	Что описывает
`ShootingRequest`	Основная сущность. UUID, внешний идентификатор `externalId`, название, описание, связь с клиентом (`@ManyToOne`, обязательна), дата съёмки, связь с локацией, статус, бюджет (числовой показатель), длительность, признак `urgent` (логический), теги, приоритет (вычисляемый), признак архива, авторы изменений.
`Client`	Клиент — заказчик съёмки (таблица `clients` из КР №1): ФИО или название организации, уникальный email, телефон.
`Location`, `LocationType`	Справочник: код, название, адрес, вместимость, тип (`PAVILION`, `STUDIO`, `OUTDOOR`, `OFFICE` — как в КР №1), признак активности.
`RequestStatus`	`NEW`, `APPROVED`, `IN_PROGRESS`, `COMPLETED`, `REJECTED`, `CANCELLED` и граф переходов (правило №6 из КР №2): методы `nextStatuses()`, `canMoveTo()`, `isFinal()`.
`PriorityLevel`	`NOT_APPLICABLE`, `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.
`RequestNote`	Заметка: заявка, автор, текст, признак закрепления.
`UserAccount`, `Role`	Учётная запись: email, имя, BCrypt-хеш, активность, обязательная смена пароля, счётчик неудач, блокировка. Роли `VIEWER` (просмотр), `OPERATOR` (редактирование), `ADMIN` (администрирование).
`RefreshSession`	Серверная refresh-сессия: SHA-256 хеш токена, срок, отзыв, ссылка на заменивший токен, устройство и IP.
`AuditEvent`	Запись журнала аудита.
`AuditedVersionedEntity`	Общий предок: `createdAt`, `updatedAt` (заполняются в `@PrePersist` и `@PreUpdate`) и `@Version` для контроля конкурентного изменения. Реализует `Persistable`: UUID назначается заранее, поэтому новизну сущности Spring Data узнаёт по пустой версии и выполняет `persist`, а не `merge`.

3.5 Входные и выходные модели: `dto/`

- `RequestDtos`: `RequestUpsert` с Bean Validation (`@NotBlank`, `@Size`, `@NotNull clientId`, `@DecimalMin` и `@DecimalMax` для бюджета, `@Min(1)` для длительности), `RequestResponse` (с данными клиента и списком `nextStatuses`), `BulkRequest`, `BulkResult`.
- `ClientDtos`: `ClientUpsert` (`@Email`, `@Pattern` для телефона `+7-XXX-XXX-XX-XX`) и `ClientResponse` с числом заявок.
- `AuthDtos`, `UserDtos`, `LocationDtos`, `NoteDtos`, `ImportDtos`, `AnalyticsDtos`, `AuditDtos`.
- `PageResponse` — страница результата: содержимое, номер, размер, количество записей и страниц.
- `ApiError` — единый формат ошибки: время, HTTP-код, код ошибки, сообщение, адрес, correlation ID, ошибки по полям.

3.6 Безопасность: `security/`

Класс	Назначение
`JwtService`	Выпускает и проверяет access JWT с подписью HMAC. Секрет из `JWT_SECRET_BASE64`, не короче 32 байт. В токене: UUID пользователя, email, имя, роли, признак временного пароля, срок жизни 15 минут.

Класс	Назначение
`JwtAuthenticationFilter`	Берёт токен из заголовка `Authorization: Bearer`, проверяет его и заново читает пользователя из базы. Отключённый пользователь не проходит, даже если токен ещё не истёк.
`PasswordChangeRequiredFilter`	Пока временный пароль не заменён, пропускает только `me`, смену пароля, `refresh` и `logout`; на остальное отвечает 403 `PASSWORD_CHANGE_REQUIRED`.
`PasswordPolicy`	Пароль от 12 до 128 символов, есть заглавная и строчная буквы, цифра и спецсимвол; не содержит имя учётной записи.
`CurrentUserService`, `UserPrincipal`	Текущий пользователь и его роли (`ROLE_VIEWER`, `ROLE_OPERATOR`, `ROLE_ADMIN`) для Spring Security.

3.7 Поиск и фильтрация: `filter/`

- `SmartFilterParser` разбирает строку поиска в типизированный `RequestFilter`. Пример: `реклама клиент:петров локация:PAV-1 статус:новая срочно:да бюджет>=200000 приоритет:высокий тег:клип архив:все`. Неизвестный статус, приоритет или бюджет вне диапазона — ошибка 400. По умолчанию показываются только активные записи.
- `RequestSpecifications` строит запрос через Criteria API: свободный текст по названию, описанию, ФИО и email клиента (join с таблицей клиентов) и внешнему ID; фильтры по клиенту, локации, статусу, срочности, приоритету, тегу (подзапрос `EXISTS`, без дублей строк), архиву; бюджет с операциями `< <= = >= >`.
- Строка пользователя никогда не склеивается с SQL или JPQL: значения передаются параметрами, символы `%` и `_` экранируются.

3.8 Ошибки: `error/`

- `BadRequestException` → 400, `AuthenticationFailedException` → 401, `NotFoundException` → 404, `ConflictException` → 409.
- `GlobalExceptionHandler` (`@RestControllerAdvice`) переводит в `ApiError` также ошибки Bean Validation (400 с полями), нехватку прав (403), конфликт версии (409), нарушение ограничения БД (409 без имени ограничения), слишком большой файл (413), неверный формат (415). Непредвиденная ошибка — 500: stack trace пишется в журнал вместе с correlation ID, клиенту уходит только общий текст.
- `AppErrorController` и `static/errors/*.html` — HTML-страницы ошибок для браузера. Адреса `/api/**` всегда отвечают JSON.

3.9 Конфигурация и web: `config/`, `web/`

Класс	Назначение
`SecurityConfig`	Цепочка фильтров без серверной сессии (`STATELESS`), открытые и закрытые адреса, Swagger только для `ADMIN`, JSON-ответы 401 и 403, CORS, заголовки безопасности (CSP, запрет фреймов), BCrypt с cost 12.
`OpenApiConfig`	Описание API для Swagger и схема авторизации `bearerAuth`.
`AppProperties`	Типизированные настройки `app.*`: JWT, администратор, CORS, флаг окончательного удаления, лимит импорта.
`BootstrapAdmin`	Первый администратор в пустой базе.
`ClockConfig`	Источник текущей даты для правила «дата съёмки не в прошлом»; в тестах его можно подменить фиксированной датой.

Класс	Назначение
`CorrelationIdFilter`	Берёт `X-Correlation-ID` из запроса или создаёт новый, кладёт в журнал (MDC) и в ответ.
`PageableFactory`	Пагинация: страница от 0, размер от 1 до 100. Сортировка только по белому списку: название, статус, бюджет, приоритет, локация, дата съёмки, дата изменения.
`RequestMetadata`	IP, User-Agent и correlation ID текущего запроса для аудита и сессий.

3.10 Миграции: `src/main/resources/db/migration/`

Таблица	Назначение и ограничения
`app_user`, `app_user_role`	Пользователи и роли. Email `UNIQUE`, роли удаляются вместе с пользователем (`ON DELETE CASCADE`).
`refresh_session`	Refresh-сессии. Хеш токена `UNIQUE`, индексы по пользователю и сроку.
`client`	Клиенты. Email `UNIQUE` и `CHECK` на нижний регистр, индекс по ФИО.
`location`	Справочник локаций, код `UNIQUE`, `CHECK` на тип и на вместимость (> 0).
`shooting_request`	Заявки. Внешние ключи на `client` (`NOT NULL`, `ON DELETE RESTRICT`), `location` и авторов; `CHECK` на статус, бюджет (0–99 999 999,99) и длительность (> 0); индексы по названию, клиенту, локации, дате, статусу, архиву, приоритету.
`request_tag`	Теги заявки, составной первичный ключ, индекс по тегу.
`request_note`	Заметки, индекс по заявке, закреплению и дате.
`audit_event`	Журнал аудита, индексы по времени, пользователю и сущности.

Файлы миграций: `V1__initial_schema.sql` — исходная схема; `V2__clients_and_location_type.sql` — таблица клиентов, перенос клиентов из уже существующих заявок (по email), тип локации и ограничения; `V3__kr1_seed_data.sql` — тестовые данные КР №1: 5 клиентов, 8 локаций, 10 заявок `KR1-001` … `KR1-010`. V3 использует `ON CONFLICT DO NOTHING` и не затирает данные, созданные вручную.

Схему меняют только новые файлы `V4__...sql` и далее. Уже применённую миграцию править нельзя: Flyway хранит её контрольную сумму в таблице `flyway_schema_history`.

3.11 Web-интерфейс для проверки: `src/main/resources/static/`

`index.html`, `app.js`, `styles.css` — страница входа и вкладки «Аналитика», «Заявки», «Клиенты», «Локации», «Пользователи», «Аудит», ссылка на Swagger. В форме заявки клиент выбирается из справочника, а список статусов содержит только текущий статус и допустимые переходы из `nextStatuses`. Access-токен хранится только в памяти вкладки, refresh-токен — в cookie `HttpOnly`, `SameSite=Strict`, которую JavaScript прочитать не может. Кнопки, недоступные роли, скрываются, но права всё равно проверяет сервер.

3.12 Конфигурация и развёртывание

Файл	Назначение
`backend/pom.xml`	Maven: Spring Boot 3.5.5, Java 21, Web, Security, Data JPA, Validation, Actuator, Flyway, PostgreSQL, JJWT, Commons CSV, springdoc (Swagger), тесты с H2.
`application.yml`	Подключение к базе из переменных окружения, `ddl-auto: validate`, Flyway, порт 8080, без stack trace в ответах, health, Swagger, параметры `app.*`.

Файл	Назначение
`.env` (не в git)	Пароль базы, JWT-секрет, администратор, порты. Шаблон без секретов — `.env.example`.
`backend/docker-compose.yml`	PostgreSQL 17 и backend отдельными сервисами, оба с healthcheck. Порты опубликованы только на `127.0.0.1` (8080 и 55432). Данные в именованном томе `framespace_postgres`: обычный `docker compose down` их не удаляет.
`start-compose.ps1`, `start-compose.sh`, `stop-compose.ps1`	Запуск и остановка; при запуске проверяется наличие `.env`.

3.13 Тесты: `src/test/java/`

Класс	Что проверяет
`ContextSmokeTest`	Весь Spring Context поднимается на H2 (профиль `test`).
`PriorityCalculatorTest`	Бизнес-правило расчёта приоритета.
`RequestStatusTest`	Граф переходов статусов: допустимые и запрещённые переходы, конечные статусы.
`EntityVersionTest`	Сохранение новой сущности возвращает тот же объект, и версия в ответе совпадает с версией в базе (иначе следующее изменение получило бы ложный 409).
`SmartFilterParserTest`	Разбор строки поиска и отказ на неверных значениях.
`PasswordPolicyTest`	Политика паролей.

4. Сквозные сценарии (удобно рассказывать на защите)

Первый вход администратора. `POST /api/auth/login` → `AuthService.login`: поиск пользователя по email, проверка BCrypt-хеша. Для неизвестного email тоже выполняется проверка с фиктивным хешем, чтобы по времени ответа нельзя было понять, есть ли такой пользователь. При неудаче `LoginAttemptService` увеличивает счётчик. При успехе выдаются access JWT и refresh-токен; в базе сохраняется только SHA-256 хеш refresh-токена. Так как у администратора временный пароль, `PasswordChangeRequiredFilter` пропускает его только к `change-initial-password`. После смены пароля доступен весь API.

Обновление токена. Access JWT живёт 15 минут. Клиент отправляет `POST /api/auth/refresh`: web-интерфейс — через cookie, Android — в теле запроса. Старая сессия отзывается, выдаётся новая пара токенов. Если кто-то предъявит уже заменённый refresh-токен, сервер считает его украденным, отзывает все сессии пользователя и пишет `REFRESH_REUSE_DETECTED` в аудит. `logout` отзывает текущую сессию.

Создание заявки. `POST /api/requests` → `@PreAuthorize` пропускает `OPERATOR` и `ADMIN` → `@Valid` проверяет `RequestUpsert` (при ошибке — 400 с полями) → `RequestService.create`: статус только `NEW`, дата не в прошлом, клиент существует, уникальность `externalId`, существование и активность локации, нормализация тегов, расчёт приоритета → `saveAndFlush` → аудит `REQUEST_CREATED` → ответ 201 с `RequestResponse`, в котором `nextStatuses = [APPROVED, REJECTED]`.

Смена статуса. `PUT /api/requests/{id}` со статусом `APPROVED` → `RequestStatus.canMoveTo` разрешает переход из `NEW` → 200. Попытка перевести `NEW` сразу в `COMPLETED` или изменить завершённую заявку → 409 «Недопустимый переход статуса». То же правило работает в массовой операции `SET_STATUS`: такие заявки попадают в список ошибок, остальные меняются. Для прошедших съёмок дата повторно не проверяется, поэтому их можно довести до конечного статуса.

Клиенты. `POST /api/clients` проверяет email и телефон, повторный email → 409. Удаление клиента со связанными заявками → 409; это же гарантирует внешний ключ `ON DELETE RESTRICT` в базе.

Изменение и конфликт версий. Клиент присылает `version`, полученную вместе с карточкой. Если за это время запись изменил другой пользователь, версии не совпадут и сервис вернёт 409. Второй уровень защиты — `@Version` в JPA.

Поиск, фильтры, сортировка, страницы. `GET /api/requests?q=...&page=0&size=25&sort=budget,desc` → `PageableFactory` проверяет размер страницы и поле сортировки → `SmartFilterParser` разбирает строку → `RequestSpecifications` строит параметризованный запрос → база возвращает одну страницу → `PageResponse`.

Архив и удаление. `DELETE /api/requests/{id}` не удаляет строку, а ставит `archived = true` с датой и автором. `POST /{id}/restore` возвращает запись. Окончательное удаление `DELETE /{id}/permanent` разрешено только `ADMIN`, только при `PERMANENT_DELETE_ENABLED=true` и только с заголовком `X-Confirm-Delete: PERMANENTLY_DELETE`.

Массовая операция. `POST /api/requests/bulk` с набором UUID (до 500) и действием `ARCHIVE`, `RESTORE`, `SET_STATUS`, `ADD_TAG` или `REMOVE_TAG`. Каждая запись обрабатывается отдельно. Ответ `BulkResult`: сколько запрошено, сколько выполнено и список ошибок по конкретным UUID.

Импорт и экспорт CSV. `POST /import/preview` проверяет файл и ничего не пишет в базу: возвращает число корректных и ошибочных строк с номером строки и полем. Проверяются и бизнес-правила: новая строка не в статусе `NEW` или недопустимый переход статуса тоже считаются ошибкой. Клиент строки ищется по `clientEmail`; если его нет, при применении он создаётся. `POST /import/apply` применяет только корректные строки в выбранном режиме. `GET /export?q=...` выгружает ту же выборку, что задана фильтром. Значения, начинающиеся с `= + - @`, экранируются, чтобы Excel не выполнил их как формулу.

Аудит. Входы и важные изменения записываются в `audit_event` в отдельной транзакции. Correlation ID связывает запись аудита, строку серверного журнала и ответ клиенту. Журнал видит только `ADMIN`.

5. Ответы на вопросы

Вопрос	Краткий ответ
Как запускается система	Docker Desktop → `.env` в корне → `start-compose.ps1` или `docker compose --env-file .env -f backend/docker-compose.yml up -d --wait` → оба сервиса healthy (`docker compose ps`) → `http://localhost:8080/actuator/health` возвращает `UP`.
Чем КР №3 связана с КР №1 и КР №2	Схема повторяет базу КР №1: клиенты, локации с типом, заявки со ссылками на клиента и локацию. Тестовые данные КР №1 загружает миграция V3. Бизнес-правила КР №2 (граф статусов, дата не в прошлом, существование клиента и локации) перенесены на сервер. JavaFX из КР №2 — отдельный настольный клиент; этот API нужен Android-приложению КР №4.
Как устроен граф статусов	`NEW` → `APPROVED` или `REJECTED`; `APPROVED` → `IN_PROGRESS` или `CANCELLED`; `IN_PROGRESS` → `COMPLETED` или `CANCELLED`; остальные конечные. Правило в `RequestStatus`, проверка в `RequestService`; недопустимый переход — 409. Клиент узнаёт допустимые шаги из `nextStatuses`.
Почему клиент — отдельная таблица	Так в КР №1: один клиент делает много заявок. Нет дублирования ФИО, email и телефона, изменение контакта видно во всех заявках, удаление клиента с заявками запрещено внешним ключом.
Зачем слои controller, service, repository	У каждого одна ответственность: HTTP и права, правила предметной области, доступ к данным. Правило меняется в одном месте и одинаково работает для web, Swagger и Android.

Вопрос	Краткий ответ
Почему контроллер не возвращает сущность	DTO фиксирует контракт API, не раскрывает лишние поля (хеш пароля, внутренние связи) и не зависит от ленивой загрузки Hibernate.
Зачем Flyway и `ddl-auto: validate`	Схема создаётся воспроизводимо и по версиям, одинаково на любой машине. Hibernate только сверяет сущности со схемой и сам таблицы не меняет.
Как устроен JWT	Три части: заголовок, данные, подпись HMAC. Сервер проверяет подпись, issuer и срок. Токен не хранится на сервере, поэтому живёт недолго (15 минут).
Зачем отдельный refresh-токен	Чтобы не входить заново каждые 15 минут. Он хранится на сервере как хеш, его можно отозвать (выход, смена пароля, отключение пользователя), при каждом обновлении он заменяется новым.
Как хранятся пароли	Только BCrypt-хеш с солью (cost 12). Пароль не хранится и не возвращается в открытом виде, в журнал не пишется.
Как защищён вход от перебора	После 5 неудач — блокировка от 5 до 40 минут; одинаковый ответ 401 для неверного email и неверного пароля; проверка хеша и для несуществующего пользователя.
Чем отличаются роли	`VIEWER` — только чтение заявок, локаций и аналитики. `OPERATOR` — плюс создание, изменение, архив, bulk, CSV, заметки. `ADMIN` — плюс справочник, окончательное удаление, пользователи, аудит, Swagger.
Где проверяются права	На сервере: `SecurityConfig` для адресов и `@PreAuthorize` для методов. Скрытие кнопок в web-интерфейсе — только удобство.
Как защищён поиск от SQL-инъекций	Строка разбирается в типизированный фильтр, запрос строит Criteria API с параметрами, сортировка — только по белому списку полей.
Как работает пагинация	На сервере: `Pageable` → `LIMIT` и `OFFSET` в базе. Размер страницы не больше 100.
Что такое optimistic locking	У записи есть `version`. Изменение проходит, только если версия совпала; иначе 409. Блокировки строк не нужны. Сервис сбрасывает изменения в базу (`flush`) до формирования ответа, поэтому клиент получает актуальную версию.
Почему архив, а не удаление	Запись, заметки и история сохраняются, ошибочное действие можно отменить. Физическое удаление — только администратором и с подтверждением.
Как обрабатываются ошибки	`GlobalExceptionHandler` → единый JSON `ApiError`; коды 400, 401, 403, 404, 409; stack trace клиенту не уходит, только в журнал вместе с correlation ID.
Где проверяются данные	Формат и диапазоны — Bean Validation в DTO; правила предметной области — в сервисе; уникальность, связи и `CHECK` — в PostgreSQL.
Зачем correlation ID	По одному идентификатору находятся ответ клиенту, строка серверного журнала и запись аудита одного запроса.
Что показывает healthcheck	Для PostgreSQL — `pg_isready`; для backend — открытый порт 8080. Backend стартует только после здоровой базы.
Почему данные не пропадают	PostgreSQL хранит файлы в именованном томе. `docker compose down` удаляет контейнеры, но не том; том удаляет только `down -v`.
Как тестируется	`cd backend && mvn test` — 16 тестов: загрузка контекста, расчёт приоритета, граф статусов, фильтр, политика паролей, совпадение версии в ответе и в базе. Тесты идут на H2, Docker для них не нужен.

6. Что показать на демонстрации

1. Запуск PostgreSQL и backend через Docker, `docker compose ps`, `/actuator/health`.
2. Вход администратором и обязательная смена временного пароля.
3. Создать пользователей с ролями `VIEWER` и `OPERATOR`; показать, что `VIEWER` получает 403 на изменение.
4. Данные КР №1 после первого запуска: 5 клиентов, 8 локаций, 10 заявок `KR1-001` … `KR1-010`.
5. Клиенты: создать клиента, получить 409 на повторный email и на удаление клиента с заявками.
6. Создание и изменение заявки; пересчёт приоритета; ошибка валидации (400), дата в прошлом (400) и конфликт версий (409).
7. Граф статусов: `NEW` → `APPROVED` проходит, `NEW` → `COMPLETED` даёт 409; в форме видны только допустимые статусы.
8. Поиск и фильтры строкой поиска (в том числе `клиент:петров`), пагинация и сортировка.
9. Архивирование и восстановление заявки.
10. Массовая смена статуса: недопустимые переходы попадают в отчёт об ошибках, остальные заявки меняются.
11. Заметки: добавить, закрепить, изменить, удалить.
12. Проверка CSV, импорт и экспорт с текущим фильтром.
13. Аналитическая сводка.
14. Управление пользователями: отключить пользователя и показать, что он не может войти; сбросить пароль.
15. Журнал аудита: поиск по событию и correlation ID.
16. Swagger UI (`/swagger-ui.html`) под администратором.
17. Миграции Flyway V1–V3 и таблицы в PostgreSQL (`localhost:55432`, таблица `flyway_schema_history`).